Laboratory 8 handout

Lab Assignment 8 : Property of operators to be used in this session.

Precedence	Associativity	Arity	Operator	Function of the operator
13	Left	binary	*, /, %	multiplicative operators
12	Left	binary	+, -	arithmetic operators
10	Left	binary	<, <=,	relational operators
9	Left	binary	==, !=	equality, inequality
5	Left	binary	&&	logical AND
4	Left	binary	||	logical OR
2	Right	binary	=, +=	assignment operators

Grammar for an assignment involving binary operators and scalar operands of "int". A symbol table is assumed to be available for semantic analysis and generation of TAC. The issues concerned with writing a syntax directed translation scheme are summarized below.

i. Given a specific programming language feature, write a context free grammar to generate all strings of interest and are also syntactically valid.

ii. Identify appropriate semantic analysis for the syntactically valid strings that are also semantically valid ; for such strings generate intermediate code as appropriate. For strings that are not semantically valid, a report on the location and nature of error has to be provided. For semanti

iii. The nature of semantic analysis indicates the attributes that are to be assigned to grammar symbols, nonterminals and terminals. These attribute values are assigned, computed and propagated during parsing. Careful choice of attributes is the first step towards performing of semantic analysis

iv. The calculation of attributes of the LHS nonterminal in terms of the attributes of the grammar symbols on the RHS is specified as semantic rules or semantic actions and executed when a reduce action is taken by the parser. This strategy of performing semantic analysis as described above is called as synthesized attributes based SDTS. In this course, we shall use synthesized attributes only.

v. Synthesized attribute based semantic evaluation is done by embedding the actions within the right hand side of productions. In the translation scheme

$A \rightarrow X_1\ X_2 \ldots X_i\ \{\textit{action}\}\ X_{i+1} \ldots X_n$

action is executed after completion of the parse for X_i. As a special case, we use the following form very frequently : $A \rightarrow X_1\ X_2 \ldots X_i\ X_{i+1} \ldots X_n\ \{\textit{action}\}$, where the ***action*** is placed at the end of the rule. This form permits the attributes of A to be defined as a function of any attributes of the symbols on the RHS of the rule.

EXAMPLE: We shall deliberately use an ambiguous expression grammar for demonstration, because as you are aware that both precedence and associativity can be specified separately for disambiguation. It is assumed that the declarations have been processed earlier and saved in a symbol table. All identifiers are of the same type, so type checking is ignored at this point.

$S \rightarrow L = E\ ;$ $\quad L \rightarrow id$

$E \rightarrow E_1 + E_2$ $\quad E \rightarrow E_1 * E_2$ $\quad E \rightarrow id$

Symbol	Attributes	Symbol	Attributes	Symbol	Attributes
id	id.place	L	L.place	E	E.place

The attribute, X.place, denotes the location for the grammar symbol X. For the symbol, "id", the attribute id.place denotes the lexeme of the identifier that is saved in the symbol table. In an expression grammar, for a nonterminal X, the attribute, X.place, is usually a compiler generated temporary where the code corresponding to X is saved.

The use of the above attributes to perform semantic analysis is illustrated with an example. Consider the C assignment : "**a = b + c * d ;**" that can be generated from the above grammar.

In the following a bottom up parser is assumed and key parsing actions along with evaluation of semantic actions and intermediate code generation. The process is best explained with actions undertaken at two stacks, parser stack and the value stack.

The state numbers are omitted from the stack.

Input	**Parser Stack**	**Value Stack**	**Actions**	**Generated Intermediate Code**
a = b + c * d ;	id	a	shift id	
= b + c * d ;	L	a	reduce id to L	
= b + c * d ;	L =	a _	shift =	
b + c * d ;	L = id	a _ b	shift id	
+ c * d ;	L = E	a _ b	reduce id to E	
+ c * d ;	L = E	a _ b	shift +	
c * d ;	L = E +	a _ b _	shift id	
* d ;	L = E + id	a _ b _ c	reduce id to E	
* d ;	L = E + E	a _ b _ c	shift *	
d ;	L = E + E *	a _ b _ c _	shift id	
;	L = E + E * id	a _ b _ c _ d	reduce id to E	
;	L = E + E * E	a _ b _ c _ d	reduce E * E to E	40 : t1 := c * d
;	L = E + E	a _ b _ c	reduce E + E to E	41 : t2 := b + t1
;	L = E	a _ b	shift ;	
	L = E ;	a _ b _	reduce L=E; to S	42 : a := t2

Successful parsing and the intermediate code generated is :

```
40 : t1 := c * d
41 : t2 := b + t1
42 : a := t2
```

Finally the grammar rules with semantic actions using the synthesized attributes defined.

S → L = E ; {emit(L.place ':=' E.place)}

L → id {L.place = id.place}

$E \rightarrow E_1 + E_2$ {E.place = newtemp(); emit(E.place ':=' E_1.place '+' E_2.place)}

$E \rightarrow E_1 * E_2$ {E.place = newtemp(); emit(E.place ':=' E_1.place '*' E_2.place)}

$E \rightarrow$ id {E.place = id.place}

- newtemp() is a function which generates a new temporary variable name, each time it is invoked.
- emit() evaluates its non-quoted arguments, concatenates the arguments in order, and returns the resulting string.
- Symbols in quotes are inserted in place after stripping off the quotes.

INTERMEDIATE CODE FORM : SYNTAX AND SEMANTICS

In Three Address Code (TAC), each statement contains 3 addresses, two for the operands and one for the result. The form is similar to assembly code and such statements may have a label, numeric or numeric. We shall use numeric labels for our experiments. The commonly used three address statements are summarized below.

Intermediate Code	TAC Syntax	Semantics
Assignment statement	n : x := y op z	op is a binary operator, whose operands are y and z and the result is saved in x and the code is labeled with a number n
	m : x := op y	op is a unary operator, whose operand is y and the result is saved in x; code is labeled with a number m
Copy statement	n : x := y	Obvious semantics
Unconditional jump	n : goto L	The three address code with label L is the next intermediate code where control flows to.
Conditional jump	n : if x relop y goto L	The relational operator, relop, is applied to x and y; control goes to code statement labeled L, if the relation is true. Else, the statement following "if x relop y goto L" is executed.
	n : if x goto L	If x is true then control goes to the code labeled L, else the code statement following "if x goto L" is executed.
Procedure or function call : p(x1, x2, …, xq)	n : param x1 n+1 : param x2 ……. n+q-1: param xq n+q : call p,q	The actual arguments are converted into a "param .. " intermediate code. The "call p" is the intermediate code corresponding to p() and the 2nd argument q denotes the number of arguments.
Array reference / assignment	x := y[i] x[j] := y	The first is used to assign to x the value in the location that is i units beyond location y, where x, y and i refer to data objects. The second one saves y in the location that is j units beyond the base address of x.
Pointer reference and assignment	x := & y	In the first form, y should be an object (perhaps an expression) that admits a l-value.

Intermediate Code	TAC Syntax	Semantics
	x := * y * x := y	In the second one, y is a pointer whose r-value is a location. The last one sets r-value of the object pointed to by x to the r-value of y.

A three address code is an abstract form of intermediate code. It can be realized in several ways, one of them is called **Quadruples**. A quadruple is a record structure with 4 fields, usually denoted by op, arg1, arg2 and result. For example, the quadruples for the expression **a + b * c** is

op	**arg1**	**arg2**	**result**	**Equivalent form**
*	b	c	t_1	$t_1 = b * c$
+	a	t_1	t_2	$t_2 = a + t_1$

where t_1 and t_2 are compiler generated temporaries. We shall use the equivalent form to generate all our intermediate code forms in our experiments.

For generating three address code, a sample implementation in the form of “tac-functions.c” is provided to you for this session. Further, C code for a naive design of symbol table has also been supplied.